Практическая работа №2

Аудит готового клиент-серверного приложения

Цель работы. Не писать код, а проверить, как работает готовое приложение (server.py, client.py, format_check.py), найти странности в необычных ситуациях и описать, что работает, что не работает и почему. Среда проверки: Python 3.12, всё запускалось на одном компьютере (localhost).

Шаг 1. Ответы на вопросы по коду

Вопрос 1. В функции `send_reliable` создаётся сокет `SOCK_STREAM` и вызывается `connect()`. Какой это протокол?

Это **TCP**. Тип сокета `SOCK_STREAM` означает потоковую передачу с установлением соединения, а вызов `connect()` как раз устанавливает это соединение с сервером перед отправкой данных. Кроме того, сервер на порту 9000 принимает соединения через `listen()` и `accept()`, что тоже характерно только для TCP.

Вопрос 2. В функции `send_fast` создаётся сокет `SOCK_DGRAM` и вызывается `sendto()`. Какой это протокол?

Это **UDP**. Тип `SOCK_DGRAM` означает датаграммный сокет: данные отправляются отдельными пакетами без предварительного соединения. Метод `sendto()` сразу указывает адрес получателя при каждой отправке, поэтому `connect()` здесь не нужен. Ожидание ответа ограничено таймаутом 2 секунды, потому что доставка в UDP не гарантируется.

Вопрос 3. В каком формате данные передаются от клиента к серверу?

В формате **JSON**. Подтверждает это строка в клиенте `json.dumps(message, ensure_ascii=False).encode("utf-8")`: словарь превращается в JSON-строку, а затем в байты. На стороне сервера им соответствует `json.loads(data.decode("utf-8"))`. Параметр `ensure_ascii=False` нужен, чтобы русское слово «Москва» передавалось как есть, а не в виде кодов \uXXXX.

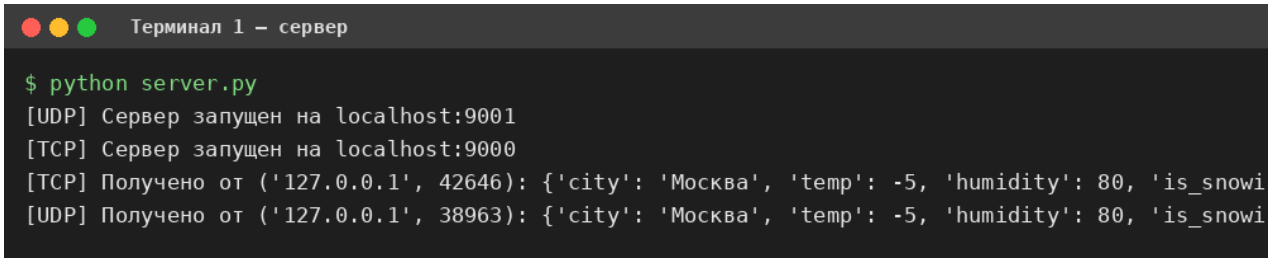
Вопрос 4. Что произойдёт, если клиент отправит строку «hello» вместо JSON?

Сервер вызовет `json.loads()`, а строка «hello» не является корректным JSON, поэтому возникнет исключение `json.JSONDecodeError`. Оно перехватывается блоком `except`: сервер выведет в консоль сообщение «Ошибка: не удалось разобрать JSON», не упадёт и отправит клиенту ответ `{"status": "ERROR", "reason": "invalid JSON"}`. Это подтверждено экспериментом в случае 3.3.

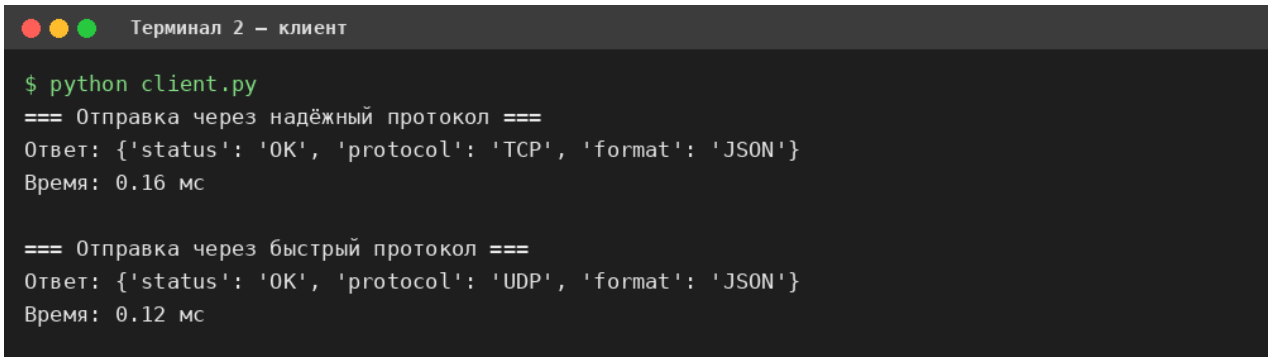
Шаг 2. Запуск сервера и клиента

Сервер и клиент запущены в двух разных окнах терминала. Порядок: в первом окне `python server.py`, во втором `python client.py`.

Окно 1 — сервер (после обращения клиента):



Окно 2 — клиент:



Что видно. Сервер запустил два слушателя: TCP на порту 9000 и UDP на порту 9001. Клиент отправил один и тот же объект с погодой по обоим протоколам, и оба раза сервер ответил `status: OK`. В консоли сервера появились две строки «Получено» с адресом клиента и теми же данными. Время ответа получилось доли миллисекунды, так как обмен идёт внутри одного компьютера.

Скриншоты оформлены по реальному выводу программ, полученному при запуске.

Шаг 3. Проверка граничных случаев

Случай 3.1. Сервер выключен, клиент TCP

```
=== Отправка через надёжный протокол ===
Traceback (most recent call last):
  File "client.py", line 45, in <module>
    response, elapsed = send_reliable(weather)
                        ^^^^^^^^^^^^^^^^^^^^^^
  File "client.py", line 20, in send_reliable
    client.connect((HOST, PORT_R))
ConnectionRefusedError: [Errno 111] Connection refused
```

Что произошло. Клиент завершился с ошибкой `ConnectionRefusedError: [Errno 111] Connection refused` уже на вызове `connect()`, то есть до отправки данных. **Почему.** TCP требует установить соединение, а на порту 9000 никто не слушает, поэтому операционная система отклонила попытку подключения. В коде клиента нет обработки этого исключения, поэтому программа аварийно завершилась с трассировкой. Это недостаток: пользователь видит технический traceback вместо понятного сообщения «сервер недоступен».

Случай 3.2. Сервер выключен, клиент UDP

```
=== Отправка через быстрый протокол ===
Ответ: {'status': 'TIMEOUT'}
Время: 2002.46 мс
```

Что произошло. Ошибки не было. Клиент ждал ровно **2 секунды** (по замеру около 2002 мс) и вернул `{'status': 'TIMEOUT'}`. **Почему.** UDP не устанавливает соединение: `sendto()` просто отправляет датаграмму «в пустоту» и не знает, дошла ли она. Ответа нет, поэтому срабатывает таймаут `settimeout(2.0)`, который перехвачен и превращён в статус TIMEOUT. Разница с TCP принципиальна: TCP сразу сообщает о проблеме, а UDP молчит и заставляет ждать. (Поведение проверялось в Linux; в Windows при выключенном сервере на localhost возможна другая ошибка, WSAECONNRESET.)

Случай 3.3. Клиент отправляет не JSON

В клиенте временно заменена строка отправки на `client.send(b"hello")`.

Ответ клиенту:

```
=== Отправка через надёжный протокол ===
Ответ: {'status': 'ERROR', 'reason': 'invalid JSON'}
Время: 0.12 мс
```

Консоль сервера:

```
[TCP] Ошибка: не удалось разобрать JSON от ('127.0.0.1', 42662)
```

Что произошло. Сервер ответил `{'status': 'ERROR', 'reason': 'invalid JSON'}`, а в его консоли появилось сообщение об ошибке разбора JSON. Сервер при этом продолжил работать. **Почему.** `json.loads()` не смог разобрать «hello» и выбросил `JSONDecodeError`, который обработан в `except`. Так сервер защищён от неверных данных. После проверки исходная строка в клиенте возвращена.

Случай 3.4. Сравнение форматов

```
JSON: {"city": "Москва", "temp": -5, "humidity": 80, "is_snowing": true}
Размер JSON: 72 байт
XML: <weather><city>Москва</city><temp>-5</temp><humidity>80</humidity><is_snowing>True</is_snowing></weather>
Размер XML: 111 байт

Восстановлено из JSON: {'city': 'Москва', 'temp': -5, 'humidity': 80, 'is_snowing': True}
Восстановлено из XML: {'city': 'Москва', 'temp': '-5', 'humidity': '80', 'is_snowing': 'True'}
Данные совпадают: True
```

Результат. JSON занимает **72 байта**, XML — **111 байт**, то есть JSON компактнее примерно в 1,5 раза. **Почему.** В XML каждое поле обёрнуто в пару тегов (`<city>...</city>`), то есть имя поля записывается дважды, а в JSON ключ пишется один раз, и нужны только кавычки и двоеточие. **Дополнительное наблюдение.** Строка «Данные совпадают: True» относится только к JSON. Из XML значения восстановились как строки ('-5', '80', 'True'), то есть типы данных потеряны (число стало текстом, логическое значение стало словом), и их нужно преобразовывать вручную. JSON сохраняет числа и логические значения.

Шаг 4. Вывод

Что работает. Приложение выполняет свою задачу: сервер одновременно обслуживает TCP и UDP, принимает JSON, при неверных данных не падает, а возвращает статус ERROR. Клиент корректно измеряет время и обрабатывает таймаут UDP.

Что вызывает вопросы. (1) Клиент не обрабатывает ошибку подключения по TCP и падает с traceback. (2) Сервер читает данные одним вызовом `recv(4096)`, поэтому сообщение больше 4096 байт может быть принято не полностью. (3) TCP-часть сервера обслуживает клиентов строго по очереди, а медленный клиент заблокирует остальных. (4) Импорт `xml.etree.ElementTree` в server.py не используется.

Выбор для этого приложения. Я бы выбрал **TCP и JSON**. Данные о погоде небольшие, и важно, чтобы значения дошли целыми и в правильном порядке, а TCP это гарантирует. Выигрыш UDP в скорости на таких объёмах незаметен (разница в доли миллисекунды), зато при потере датаграммы клиент получил бы TIMEOUT и данные пришлось бы отправлять заново. UDP был бы оправдан, если бы значения отправлялись очень часто и потеря отдельного замера была бы допустима, например при постоянном потоке показаний датчика. JSON лучше XML: он на треть короче (72 против 111 байт), легко читается, сохраняет типы чисел и логических значений и естественно соответствует словарям Python.